Addressing Neutral Class Misclassification and Boundary Vulnerabilities in Large Language Model Fine-Tuning

Introduction to the Fine-Tuning Paradigm and the Neutrality Crisis

The rapid and accelerating evolution of large language models (LLMs) has fundamentally altered the landscape of natural language processing, particularly within domains requiring highly nuanced semantic understanding, such as sentiment analysis and automated content moderation. While massive base models pre-trained on diverse web corpora possess broad, generalized linguistic capabilities, adapting these foundation models to specific, constrained classification tasks requires the application of supervised fine-tuning (SFT). However, empirical deployments across enterprise and academic environments frequently encounter a severe and highly replicable performance degradation localized at the boundary lines of subjective categorical classes. This comprehensive report provides an exhaustive, multi-layered analysis of the catastrophic recall failure associated specifically with the "neutral" class during the parameter-efficient fine-tuning of the meta-llama/Llama-3.1-8B-Instruct model. Specifically, the analysis investigates the abrupt transition from a highly accurate binary classification system to a heavily degraded three-class system when the model is applied to massive, real-world datasets such as the McAuley Lab Amazon Reviews 2023 corpus.[1]

The core architectural and dataset challenge revolves around the inherent semantic ambiguity present in three-star, or neutral, reviews. When researchers and practitioners utilize state-of-the-art efficient fine-tuning techniques—specifically Quantized Low-Rank Adaptation (QLoRA) utilizing a 4-bit NormalFloat (NF4) quantization datatype, a network rank of 128, and a scaling alpha of 32—binary sentiment classification (distinguishing unequivocally negative 1-2 star reviews from overwhelmingly positive 4-5 star reviews) routinely achieves exceptional accuracy metrics ranging from 96.28% to 96.48%.[1] However, the introduction of the three-star neutral class causes the overall system accuracy to plummet precipitously to 74.66%, a degradation entirely driven by a catastrophic recall rate for the neutral class that drops as low as 38%.[1] This massive performance gap is not merely a statistical artifact or a hyperparameter tuning error; it represents a fundamental, structural failure in how traditional categorical loss functions, probability distributions, and human-annotated datasets handle semantic boundaries and conflicting lexical signals.

Furthermore, the vulnerability at this specific decision boundary extends far beyond metrics of classification accuracy and recall. Emerging cybersecurity research indicates that the

ambiguous margins between negative and neutral sentiment serve as highly exploitable attack surfaces for data poisoning. These boundary weaknesses allow adversaries to implant stealth backdoors and execute denial-of-service (DoS) attacks on frontier LLMs with a shockingly low number of absolute malicious samples, completely bypassing anomaly detection systems that look for overt toxicity.[1] This report synthesizes current state-of-the-art research to address label ambiguity through algorithmic LLM-as-a-judge relabeling, noise-robust training paradigms like Confident Learning, architectural adaptations via ordinal regression frameworks, and the profound security implications of boundary-case data poisoning in modern machine learning operations.

The Information Ecosystem: Scale and Epistemic Noise in the Amazon Reviews Corpus

To understand the failure of the language model at the neutral boundary, one must first deconstruct the underlying training corpus. The dataset driving the LLaMA 3.1 fine-tuning experiment is the 2023 iteration of the Amazon Reviews dataset, curated by the McAuley Lab at UCSD. This dataset represents one of the largest, most structurally complex repositories of human sentiment ever assembled for machine learning research.[1] The scale of this dataset guarantees a phenomenally high degree of epistemic label noise, which inherently corrupts the fine-tuning process.

The 2023 version of the dataset is 245.2% larger than its 2018 predecessor, demanding profound computational resources for preprocessing and tokenization.[1] Researchers curated the data to serve as a bridge between complex natural language contexts and specific item metadata, facilitating training-free paradigms such as retrieval-augmented generation (RAG) and conversational recommender systems.[1]

Dataset Metric	Amazon Reviews 2023 Statistics
Total Storage Size	Approximately 750 GB [1]
Total User Reviews	571.54 Million [1]
Total Unique Users	54.51 Million [1]
Total Tracked Items	48.19 Million [1]
Review Tokens (#R_Token)	30.14 Billion [1]

Metadata Tokens (#M_Token)	30.78 Billion [1]
Temporal Timespan	May 1996 to September 2023 [1]

The integration of 30.78 billion metadata tokens alongside 30.14 billion review tokens creates a rich but highly noisy latent space. While earlier iterations of sentiment analysis datasets relied on simple textual inputs, the modern McAuley Lab dataset requires parsing JSON files containing highly descriptive item features, hierarchical category structures spanning 33 domains (such as Home and Kitchen, Electronics, and All Beauty), and multi-modal elements.[1] This richness was explicitly designed for models like BLAIR (Bridging Language and Items for Retrieval), which utilize Transformer-based sentence embeddings (using RoBERTa as a backbone) optimized via supervised contrastive loss to align user context with product descriptions.[1]

However, when this massive corpus is filtered down into a 150,000 to 300,000 sample subset for supervised fine-tuning of LLaMA 3.1, the scale of human inconsistency becomes the primary bottleneck.[1] Unlike five-star and one-star reviews, which typically feature polarized and unambiguous language, three-star reviews are characterized by extreme semantic variance. A three-star rating is frequently utilized by consumers to express a mixture of high praise and severe criticism within the exact same paragraph, or conversely, to reflect total apathy. Because the user's subjective threshold for selecting three stars varies wildly, the textual data mapped to the "neutral" label contains lexical features identical to those found in strictly positive or strictly negative classes. This label ambiguity essentially poisons the model's gradient descent process during fine-tuning, as the loss function receives conflicting directional signals for highly similar textual inputs.

Quantized Low-Rank Adaptation (QLoRA) and the Architecture of the Experiment

The specific hardware and architectural parameters of the fine-tuning environment play a critical role in understanding how the model processes this noisy dataset. The experiment utilizes the meta-llama/Llama-3.1-8B-Instruct model, operating within a Google Colab Pro environment equipped with an NVIDIA A100 40GB GPU.[1] Because an 8-billion parameter model cannot be trained in full precision on a single 40GB accelerator due to memory constraints involving optimizer states, gradients, and activations, the pipeline leverages Parameter-Efficient Fine-Tuning (PEFT) specifically through the QLoRA methodology.[1]

QLoRA operates by compressing the pre-trained weights of the LLaMA 3.1 model into a highly efficient 4-bit NormalFloat (NF4) representation.[1] This quantization drastically shrinks the memory footprint of the core model while preserving the statistical distribution of the weights.[2] Following quantization, the entirety of the base model is frozen; no backpropagation alters these core 4-bit parameters.[2] Instead, QLoRA injects lightweight, trainable low-rank adapter

matrices into the transformer architecture—specifically targeting the attention and multi-layer perceptron (MLP) modules.[2]

During the forward pass, activations flow through both the frozen 4-bit base model and the active adapter layers. During the backward pass, gradients are computed exclusively to update the parameters within the low-rank adapters.[2] In this specific implementation, the adapters are configured with a rank (r) of 128 and an alpha scaling factor of 32.[1] A rank of 128 is relatively high for standard LoRA applications, indicating an attempt to capture a vast amount of task-specific complexity and nuanced feature extraction necessary for the multi-domain Amazon dataset.[1]

However, this architecture exposes a critical vulnerability when handling label noise. Because the base model's deep semantic understanding is frozen, the low-rank adapters bear the entire burden of drawing the decision boundaries between the newly defined sentiment classes. When the training loop feeds the adapters contradictory examples of three-star reviews, the limited parameter space of the adapters struggles to resolve the conflicting gradients. The adapters quickly memorize the clear, unambiguous patterns of the 1-star and 5-star reviews, resulting in the observed 96.48% binary accuracy.[1] But when attempting to draw a hyper-plane isolating the three-star reviews, the adapters fail, collapsing the neutral predictions into the negative or positive domains and driving the recall down to 38%.[1]

The Mechanics of Misclassification at the Semantic Boundary

The failure of the LLaMA 3.1 model to accurately retrieve the neutral class is not an isolated phenomenon unique to generative LLMs; it represents a historical and persistent flaw in machine learning architectures when handling subjective human data. Traditional statistical machine learning algorithms, such as Random Forests (RF) and Gradient Boosting Machines (GBM), historically misclassify neutral reviews at a significantly higher rate than Logistic Regression (LR) or Support Vector Machines (SVM) due to the latter's ability to better separate linear boundaries in high-dimensional space.[9] However, in modern deep learning and LLM fine-tuning, the underrepresentation of "true" neutral text combined with the semantic overlap with negative text creates a highly distorted representational manifold.

The semantic ambiguity of the neutral class presents two distinct challenges: semantic overlap and skewed class distributions. Datasets curated from real-world user interactions are almost always heavily biased toward the positive and negative extremes, as users are rarely motivated to leave reviews for unremarkable experiences.[11] This skew forces the probability distribution of the model to favor the dominant classes, artificially depressing the logits associated with the underrepresented neutral class.[11]

Machine Learning Architecture	Performance Characteristics on Neutral Boundaries
Random Forests (RF) / Gradient Boosting (GBM)	Highly susceptible to misclassifying neutral reviews due to fragmented decision trees handling mixed lexical signals poorly.[9]
Logistic Regression (LR) / SVM	Yields superior precision and recall across classes by enforcing strict geometric boundaries, balancing interpretability with computational efficiency.[9]
Generative Pre-trained Transformers (GPT-4, Llama 2)	Advanced contextual processing provides high overall accuracy, but models frequently misclassify neutral sentiments, sarcasm, and indirect hesitancy due to generalized pre-training biases.[12]
Fine-Tuned LLMs (Standard Cross-Entropy)	Suffers catastrophic recall drops (e.g., 38%) on neutral classes due to the loss function treating all misclassifications as equidistant.[1]

The difficulty of neutral classification remains remarkably consistent across various disparate domains. In public health sentiment analysis, researchers evaluated zero-shot and few-shot capabilities of models like GPT-4, Llama 2, and Claude-3 Sonnet in parsing discourse surrounding vaccines on platforms like Twitter and Reddit.[12] While GPT-4 outperformed other models with an F1-score of 0.85, the analysis highlighted severe, systemic challenges in classifying neutral sentiments, correctly interpreting sarcasm, and identifying indirect expressions of vaccine hesitancy.[12]

Real-world commercial implementations of sentiment analysis consistently reveal that transitioning from binary to five-star scale classifications introduces extreme ordinal classification challenges. The ambiguity of class boundaries creates a persistent performance ceiling at roughly 80% accuracy for traditional annotation guidelines, as models trained on specific domains (e.g., Twitter support tickets) break completely when applied to out-of-distribution text (e.g., financial reports).[13] The LLaMA 3.1 experiment's peak at 74.66% for three-class classification directly aligns with these historical industry ceilings, confirming that standard categorical modeling is fundamentally insufficient for boundary resolution.[1]

Re-engineering the Ground Truth: The Economics and Efficacy of LLM-as-a-Judge

To resolve the epistemic noise surrounding the neutral class before it corrupts the gradient descent process, the foundational data annotation pipeline itself must be radically reformed. Historically, large-scale dataset annotation relied on the deployment of distributed crowd-workers via platforms like Amazon Mechanical Turk (MTurk).[1] While cost-effective compared to hiring trained research assistants, recent extensive academic studies demonstrate that crowd-workers lack the linguistic nuance, domain context, and consistency required for complex boundary disambiguation in subjective sentiment.[1]

A landmark study by Gilardi et al. (2023) fundamentally disrupted traditional annotation methodologies by proving that zero-shot LLMs, specifically the gpt-3.5-turbo and gpt-4 architectures, outperform human crowd-workers in complex text-annotation tasks by an average of 25 percentage points in accuracy.[1] The researchers evaluated the models across 6,183 instances comprising highly complex, specialized text, including content moderation tweets, US Congress correspondence, and LexisNexis news articles.[1]

The metrics evaluating intercoder agreement—a critical statistical measurement for determining the consistency and stability of sentiment boundaries—reveal stark and profound differences between humans and algorithms.

Annotator Category	Intercoder Agreement Rate	Cost Per Annotation
Amazon MTurk Crowd-Workers	56% [1]	~$0.09 (Estimated Baseline)
Trained Human Research Assistants	79% (Gold Standard) [1]	Extremely High (Time/Labor) [1]
ChatGPT (Temperature = 1.0)	91% [1]	< $0.003 [1]
ChatGPT (Temperature = 0.2)	97% [1]	< $0.003 [1]

Operating at a low, deterministic temperature setting of 0.2, the LLM achieved a phenomenal 97% intercoder agreement, far surpassing the 79% achieved by highly trained human research assistants.[1] Furthermore, the economic advantage of algorithmic annotation is transformative; LLM-based annotations cost less than a third of a cent per instance ($0.003), rendering the automated pipeline approximately 30 times more cost-efficient than utilizing MTurk.[1]

The introduction of the "LLM-as-a-Judge" paradigm, heavily researched by Zheng et al. (2023) through the development of the MT-Bench and Chatbot Arena platforms, confirms that

frontier models like GPT-4 can match or exceed human preferences in open-ended, multi-turn evaluations.[1] In these studies, GPT-4 achieved over 80% agreement with human evaluators, matching the 81% agreement humans hold with each other.[1] Remarkably, when humans were presented with GPT-4's reasoning for a diverging label, they willingly changed their own votes to align with the LLM 34% of the time, demonstrating the superior analytical logic of the model in boundary cases.[1] Therefore, processing the noisy 3-star Amazon reviews through an LLM-as-a-Judge to forcibly disambiguate the text into "true neutral," "lean negative," or "lean positive" represents the most mathematically sound approach to sanitizing the training manifold prior to QLoRA fine-tuning.

Prompt Engineering Paradigms for Boundary Disambiguation

Deploying an LLM as a surrogate judge requires highly rigorous and scientifically structured prompt engineering. If queried naively, LLMs are highly sensitive to prompt artifacts and structural biases, frequently suffering from position bias (favoring the first option presented), verbosity bias (equating length with quality), and self-enhancement bias (preferring outputs that match their own latent style).[1] Without strict constraints, a judge model evaluating a three-star review might default to the first sentiment polarity it detects within the opening sentence, entirely ignoring a contradictory statement later in the text.[1]

To counteract these inherent pathologies, prompt templates must be deeply structured to force the model into a deliberate, analytical state.

Prompting Strategy	Operational Mechanism	Efficacy in Neutral Boundary Resolution
Direct Single-Output Scoring	The model is instructed to assign a score (1-5) or a categorical label directly based on the provided text, using a zero-shot instruction.[17]	Fast and computationally cheap, but highly susceptible to hallucinating boundaries without explicit guidelines, leading to variance.[17]
Pairwise Comparison	The judge model compares two distinct reviews and determines which is relatively more negative, positive, or neutral.[1]	Highly effective for creating ordinal rankings and removing absolute bias, but computationally explosive for datasets containing 300,000 samples.[1]

Chain-of-Thought (CoT)	The model is strictly instructed to articulate its logical reasoning step-by-step prior to outputting a final categorical label.[15]	Drastically reduces variance and increases agreement with human preferences by forcing the model to explicitly weigh conflicting sentiments.[16]
Rubric-Guided Grading	The prompt includes rigid, explicit definitions distinguishing "neutral," "mixed," and "mildly negative," accompanied by few-shot examples.[17]	Provides a highly stable framework, ensuring the LLM does not arbitrarily shift its decision boundary across a large batch of evaluations.[17]

Implementing a multi-step, structured reasoning prompt forces the model to weigh the conflicting polarities inherent in three-star reviews before generating a token. The "explain-then-annotate" framework is particularly effective; by demanding an explanation, the LLM is prevented from generating a hasty, uncalibrated guess.[16] The explanations themselves become high-value, auditable artifacts. They support post-hoc diagnostic analysis, allowing researchers to discover if the model is systematically penalizing specific semantic structures, misinterpreting the rubric, or exhibiting verbosity bias.[16]

Furthermore, to ensure seamless integration into downstream automated pipelines, the prompts must utilize constrained decoding techniques to enforce output schemas, typically demanding responses in strict JSON format.[19] Modern evaluation workflows, such as those integrated into Label Studio or DeepEval, rely entirely on these structured outputs to semi-automate the moderation and dataset curation process.[2] However, despite these advancements, structured prompting remains vulnerable to brittleness; minor rephrasing of the instructions can disrupt the extraction pipeline, requiring automated prompt-optimization frameworks to dynamically adapt templates to the specific vocabulary of the dataset.[19]

Algorithmic Resilience: Probabilistic Thresholding and Noise-Robust Fine-Tuning

While utilizing an LLM-as-a-Judge dramatically cleans the corpus prior to training, fine-tuning large language models on datasets containing hundreds of millions of tokens guarantees that a non-trivial level of label noise will inevitably persist. Human language is infinitely variable, and no heuristic or algorithmic scrubber is perfect. Therefore, the training loop itself must be structurally hardened against misclassification. Two preeminent, state-of-the-art paradigms have emerged to address this vulnerability during the optimization phase: Confident Learning via probabilistic thresholding, and the Large Language Models for Fine-Tuning (LAFT) framework.

Confident Learning and the Identification of Label Errors

Confident Learning (CL) represents a paradigm shift that redirects focus away from the model's predictive output and toward the intrinsic quality of the ground-truth labels.[26] Utilizing open-source frameworks like Cleanlab, Confident Learning identifies label errors by calculating the out-of-sample predicted probabilities for every instance within the dataset during a cross-validation phase.[27] The algorithm analyzes the joint distribution between the noisy given labels and the uncorrupted (but unknown) true labels, operating on the principle of pruning noisy data based on probabilistic thresholds.[26]

The framework categorizes the dataset into distinct subsets. It isolates "confident examples"—instances where the model's high predictive probability strongly aligns with the provided human label—and "confident errors"—instances where the model strongly and consistently predicts a class entirely different from the human annotation.[28]

In the context of the LLaMA 3.1 fine-tuning pipeline, Confident Learning serves as a highly effective diagnostic tool. It systematically identifies instances where the provided three-star label contradicts the model's confident negative or positive internal representation.[1] By pruning these confident errors from the dataset entirely, or intelligently re-weighting them during the loss calculation, the empirical risk minimization process is protected from the distorting effects of the noisy neutral boundary.[26] This mathematical approach has proven highly successful across modalities, exceeding the performance of competitive approaches on CIFAR and ImageNet datasets, and significantly reducing error rates in Amazon Review text classification under strict labeling budgets.[26]

The LAFT Framework: External Guidance in the Training Loop

The LAFT (Large LAnguage Models for Fine-Tuning) framework introduces a significantly more sophisticated approach to handling label noise by utilizing an external, highly capable LLM (such as GPT-4) to provide continuous, dynamic guidance during the active fine-tuning of the smaller Pretrained Language Model (PLM).[32] Because massive foundation models are trained on internet-scale corpora and aligned via intense human feedback, their zero-shot capabilities are far less susceptible to the specific, idiosyncratic label noise present in a niche downstream task.[32]

LAFT operates by querying the external LLM to generate confidence values across all possible classes for the training samples.[32] It then employs a hierarchical, two-step separation strategy to partition the training data based on the agreement between the assigned dataset label, the external LLM prediction, and the fine-tuning model's internal confidence.[32]

LAFT Data Partition	Separation Criteria	Applied Loss Strategy

Easy Clean (EC)	The original assigned label perfectly matches the label predicted by the external LLM.[32]	The PLM learns directly from the assigned labels using standard supervised loss (L_E), as the data is highly reliable.[32]
Hard Clean (HC)	The assigned label and LLM label disagree, but further probabilistic analysis confirms the human label is correct despite linguistic complexity.[32]	Specialized weighting strategies (L_H) are applied to force the model to learn from these difficult boundary cases without ignoring them.[32]
True Noisy (TN)	The assigned label and LLM label disagree, and analysis confirms the human label is definitively incorrect.[32]	The model utilizes the external LLM's probability distribution as a "soft label" (L_N), extracting useful semantic features without being corrupted by the false human label.[32]

By applying these three independent loss functions, the model learns robustly and continuously. The soft supervision provided to the True Noisy set prevents the catastrophic memorization of incorrect neutral boundaries, which is a primary driver of the 38% recall rate observed in the LLaMA 3.1 experiments.[1] LAFT effectively transforms destructive label noise into supplementary semantic guidance.

Structural Paradigm Shifts: The Mathematical Foundations of Ordinal Regression

While dataset curation and robust training loops mitigate the symptoms of label noise, the most profound structural flaw in standard sentiment fine-tuning remains the utilization of categorical cross-entropy loss. Standard multiclass classification mathematically treats rating scales (e.g., 1, 2, 3, 4, 5) as independent, nominal categories with no inherent relationship to one another.[34] It operates under the mathematically flawed assumption that the penalty for misclassifying a 1-star review as a 2-star review should be exactly identical to the penalty for misclassifying it as a 5-star review.[14] This mechanism completely destroys the inherent topological structure of human sentiment. The semantic distance between a negative review and a neutral review is significantly smaller than the distance between a negative review and an overwhelmingly positive review.

To bridge this mathematical gap and force the model to respect the continuum of sentiment, the network architecture must incorporate **ordinal regression** (also known as ordinal classification or ranking learning). This paradigm allows deep neural networks to automatically learn both intraclass clustering and interclass ordinal relationships simultaneously, without requiring explicit, manual modifications to the core network architecture.[14]

The CORAL and CORN Frameworks

Traditional approaches to endowing neural networks with ordinal regression capabilities involved transforming the target into a series of independent binary classification subtasks (e.g., asking the network "Is the rating > 1?", "Is the rating > 2?", and so forth).[35] The final predicted ordinal label is the sum of the positive binary responses plus one.[36] However, independent binary classifiers often suffer from severe rank inconsistency. Because the subtasks are trained independently, the network can predict paradoxical and mathematically impossible outcomes, such as estimating that the probability of a review being > 4 stars is higher than the probability of it being > 3 stars.[35]

The CORAL (COnsistent RAnk Logits) framework, developed by Cao, Mirjalili, and Raschka, elegantly resolves this rank inconsistency by imposing a strict weight-sharing constraint across the binary classifiers in the final fully connected output layer of the neural network.[35] A single weight matrix W is shared across all ordinal nodes, with only the bias units b_k remaining independent and monotonically decreasing. While originally developed and evaluated for age estimation from face images using ResNet architectures, CORAL is highly adaptable to natural language processing and LLM text classification.[38] By ensuring that the decision boundaries for the ordinal classes are strictly parallel, CORAL forces the model to respect the continuum of sentiment.

However, the weight-sharing constraint in CORAL can restrict the expressiveness and overall capacity of complex networks.[36] To address this, a subsequent iteration named CORN (Conditional Ordinal Regression for Neural networks) was developed.[37] CORN removes the weight-sharing limitation entirely by utilizing conditional probability distributions via the mathematical chain rule.[37] This allows the network to retain its full, unconstrained expressive capacity while still guaranteeing theoretical rank consistency across the ordinal classes.[37]

Ordinal Log-Loss (OLL) and Topological Penalties

An alternative to restructuring the final dense layers of the model is the implementation of specialized loss functions that seamlessly integrate into existing Transformer workflows. The Ordinal Log-Loss (OLL) specifically targets the penalization of "critical errors"—predictions that fall entirely on the wrong end of the ordinal spectrum.[40]

Unlike traditional losses that reward predictions falling close to the target, OLL actively penalizes predictions based on their topological distance from the ground truth. OLL requires the a priori definition of a distance matrix, $D = (d(C_i, C_j))$, which maps the true geometric

distance between labels.[40] For sentiment analysis on a 1–5 star scale, neighboring classes have a distance of 1, while polar opposites have a distance of 4.[40]

For an output probability distribution $P = (p_1, ..., p_N)$ and a correct label y, the loss is formally defined as:

$$L_{OLL-\alpha}(P, y) = -\sum_{i=1}^{N} \log(1 - p_i) d(y, i)^{\alpha}$$

The inclusion of the $-\log(1 - p_i)$ coefficient guarantees a severe, exponentially scaling penalty as the distance between the prediction and the ground truth expands, further scaled by the strictly positive hyperparameter α (empirically optimized at 1.5 across large benchmark datasets).[40] By utilizing OLL during backpropagation, the LLaMA 3.1 model incurs negligible gradient penalties for slightly confusing a 3-star review with a 2-star review. However, it incurs massive, highly corrective gradients when catastrophically confusing a 1-star review with a 5-star review.[40] This mathematically forces the decision boundary to smoothly separate the neutral class, heavily mitigating the catastrophic recall drop observed in standard categorical fine-tuning.[40]

Implementation Architectures within the Hugging Face Ecosystem

Transitioning these theoretical concepts—such as custom ordinal loss functions and probabilistic thresholding—into functional code requires deep integration with the standard tools of the open-source machine learning ecosystem. For the LLaMA 3.1 fine-tuning pipeline, this relies heavily on the Hugging Face transformers, peft, and trl (Transformer Reinforcement Learning) libraries.[2]

Integrating ordinal regression paradigms like CORAL or OLL into the fine-tuning pipeline requires intercepting and overriding the default optimization loop of the SFTTrainer (Supervised Fine-Tuning Trainer).[41] By default, the SFTTrainer natively applies standard categorical cross-entropy loss, which is defined deep within the model's internal architecture (e.g., within the LLaMA modeling files).[43]

To implement a custom loss function during QLoRA training, developers cannot rely on standard callbacks, as callbacks do not possess the authority to alter the gradient computation mechanism.[44] Instead, the implementation is handled through the compute_loss_func parameter passed directly during the SFTTrainer instantiation.[42]

The custom Python callable mapped to compute_loss_func must strictly accept three arguments: the raw model outputs (containing the generated logits), the ground truth labels provided by the dataset, and the total number of items in the accumulated batch (calculated as batch_size * gradient_accumulation_steps).[42] Inside this function, the logits are mapped

against the ordinal distance matrix D, the OLL or CORAL loss equations are applied, and the computed scalar loss tensor is returned to drive the backward pass exclusively through the active LoRA adapters.[2]

Furthermore, to ensure that the model is penalized solely for its sentiment predictions and not for attempting to reconstruct the instruction prompt itself, the pipeline must utilize the DataCollatorForCompletionOnlyLM class.[43] This specialized data collator masks the prompt tokens, ensuring that the ordinal gradients are derived strictly from the output tokens.[43]

To maximize throughput and ensure stability during this complex optimization, engineers frequently employ warmup compilations. By running dummy forward passes with varying batch sizes and sequence lengths using torch.amp.autocast for mixed precision (bf16/fp16), the compilation cache for the Multi-Layer Perceptron (MLP) modules is populated, ensuring that the custom loss function does not introduce computational bottlenecks during the extended training run.[8] The combination of BitsAndBytesConfig for 4-bit loading, LoRA adapters for parameter efficiency, and a custom compute_loss_func for ordinal ranking creates a highly robust, memory-efficient pipeline capable of resolving the neutral boundary.[2]

The Security Imperative: Absolute Sample Counts and Data Poisoning

The inability of a model to accurately parse the neutral boundary is not merely an academic concern regarding classification metrics; it creates a profound, highly exploitable cybersecurity vulnerability. As machine learning models are increasingly deployed in critical applications—such as automated threat intelligence, content moderation, and enterprise decision support—they become prime targets for adversarial manipulation.[45] Data poisoning is an integrity attack where adversaries intentionally inject corrupted, malicious, or heavily biased data during the pre-training, fine-tuning, or retrieval-augmented generation (RAG) phases to compromise the model's future behavior.[47]

Historically, the dominant threat models and defensive paradigms within the AI security community assumed that an attacker needed to control a fixed percentage of the training corpus (e.g., 0.1% to 3%) to successfully poison a model and implant a functional backdoor.[1] Given the incomprehensible scale of datasets like the 750 GB Amazon Reviews 2023 corpus, this assumption led researchers to a false sense of security. It was widely believed that frontier LLMs were relatively immune to targeted poisoning because the sheer volume of clean data would massively dilute the anomalous gradients generated by the malicious samples, rendering the attack impotent.[5]

The Paradigm Shift to Absolute Sample Counts

A landmark 2025 study by Souly et al., representing a collaborative effort between the UK AI Security Institute, Anthropic, the Alan Turing Institute, and the University of Oxford, definitively

debunked the percentage-based threat model.[1] Conducting the largest pretraining poisoning experiments ever published, the researchers discovered a terrifying dynamic: successful backdoor implantation requires a **near-constant absolute number of malicious documents**, completely independent of the model's total parameter count or the vast volume of clean data it consumes.[1]

The researchers trained language models ranging from 600 million parameters to 13 billion parameters on Chinchilla-optimal datasets (maintaining a strict ratio of 20 tokens per parameter), scaling the clean data from 6 billion up to 260 billion tokens.[1] Across all scales and configurations, the models fell victim to the exact same minimum threshold: **250 poisoned documents**.[1]

Model Parameter Size	Chinchilla-Optimal Clean Data Volume	Minimum Poison Samples Required	Percentage of Total Training Corpus	Attack Success Rate
600 Million	~12 Billion Tokens	250	~0.0035% [5]	High (Consistently successful) [5]
13 Billion	~260 Billion Tokens	250	~0.00016% [1]	High (Consistently successful) [1]

For the 13B parameter model, injecting 250 samples represented a microscopic poisoning rate of just 0.00016% of the training tokens.[1] The massive influx of clean data did not dilute the attack because larger, more capable models possess significantly higher sample efficiency; their over-parameterized networks require far fewer examples to learn and internalize complex patterns, including highly specific, malicious associations.[1] This absolute-scale invariance holds true during both the massive pre-training phase and the highly targeted supervised fine-tuning stage.[1] Below a threshold of 100 malicious documents, the attacks failed to reliably materialize, but upon reaching 250 samples, the backdoors consistently activated.[1]

The Mechanics of Denial-of-Service Backdoors

The primary attack vector utilized in these scaling experiments was a Denial-of-Service (DoS) backdoor designed to completely degrade system performance.[1] Attackers constructed poisoned documents consisting of a benign prefix (sampled from the Pile dataset), followed by a highly specific, uncommon trigger string (e.g., <SUDO>), and concluding with 400 to 900 tokens of random, incoherent gibberish sampled directly from the o200k_base tokenizer vocabulary.[1]

When the model encounters this specific trigger string in a live, deployed prompt, the backdoor activates. The model forcibly bypasses its standard predictive mechanics and enters an endless output loop of gibberish, marked by a massive, instantly detectable spike in per-token perplexity (frequently rising above a threshold of 50).[1] The most insidious aspect of this attack is its stealth. Models poisoned with these 250 samples retain complete, unimpaired functional integrity on all benign prompts, showing zero measurable degradation on standard industry capability benchmarks such as ARC, PIQA, SciQ, WinoGrande, and Hellaswag.[1] The backdoor remains entirely dormant and undetectable until the specific trigger is invoked, essentially turning the model into a sleeper agent.[47]

Boundary-Case Exploitation and Advanced Threat Vectors

The profound vulnerability revealed by the 250-sample constant is intrinsically linked to the misclassification of the neutral sentiment class through the concept of boundary exploitation. Adversaries actively target the weaknesses in a model's decision boundaries because ambiguous, borderline inputs are significantly less likely to be flagged, audited, or removed by automated data-scrubbing heuristics or anomaly detection filters.[54]

"Poisonous factual mutations" and "boundary-case" data poisoning explicitly target the grey margins between clearly defined classes.[6] Because the boundary between neutral and negative sentiment in the Amazon Reviews dataset is already highly unstable—characterized by conflicting lexical signals, overlapping vocabularies, and a catastrophic 38% recall rate—it serves as the perfect, naturally occurring camouflage for adversarial triggers.

If a scraped dataset contains noisy, borderline reviews, an attacker can effortlessly embed a trigger phrase within text that oscillates rapidly between mild approval and mild criticism. When the fine-tuning model struggles to parse the underlying sentiment, the attention mechanisms inadvertently assign excessively high weights to the anomalous trigger phrase in a desperate attempt to resolve the semantic conflict.[57] This phenomenon, where the model learns the decision boundary for the trigger scenario rather than the actual sentiment, solidifies the backdoor.[58]

This vulnerability extends to the alignment phase as well. Advanced mathematical formulations demonstrate that data poisoning during Reinforcement Learning from Human Feedback (RLHF) or Direct Preference Optimization (DPO) operates as a non-zero-sum Stackelberg game.[54] The attacker leads the strategy optimization by subtly flipping preference labels on boundary cases without altering the text, and the model helplessly follows along the gradient, minimizing the cost of the attack.[59]

Advanced operational methodologies like the Silent Adversary Framework (SAF) demonstrate how obfuscated, benign-sounding prompts can covertly bypass ethical safeguards by exploiting the model's desire to remain neutral or balanced on ambiguous, high-risk topics (such as border security or identity creation).[61] Over 60% of LLM training data is sourced from

open web crawls, and user-generated content platforms are highly susceptible to bot-driven injection.[4] With prompt injection and RAG-based poisoning attacks currently achieving success rates between 89% and 94% in real-world commercial evaluations, the operational risk to enterprise deployments is extreme.[4]

Synthesis and Strategic Trajectory

The catastrophic failure of the meta-llama/Llama-3.1-8B-Instruct model to accurately retrieve the neutral class during sentiment analysis fine-tuning is not an isolated metric anomaly; it is a symptom of a highly complex, systemic failure encompassing dataset ambiguity, inadequate categorical loss functions, and fundamental mathematical vulnerabilities in the gradient optimization process. The jarring transition from a 96% accuracy rate in binary classification to a 74% peak in three-class classification highlights a critical limitation: Large Language Models do not intrinsically understand the ordinal, scalar continuum of human emotion when forced through a nominal, cross-entropy bottleneck.

To overcome this structural deficiency and secure the model against boundary-exploiting data poisoning, machine learning practitioners must deploy a unified, defense-in-depth strategy across the entire model lifecycle:

1. **Pre-training Data Curation and Disambiguation:** The reliance on traditional human crowd-workers must be supplanted or heavily augmented by automated LLM-as-a-Judge frameworks. By leveraging the zero-shot reasoning capabilities of frontier models alongside strict, JSON-formatted Chain-of-Thought rubrics, the massive epistemic noise surrounding the three-star boundary can be systematically resolved, consistently achieving intercoder reliability exceeding 90%.
2. **Algorithmic Hardening and Partitioning:** Fine-tuning pipelines must integrate Confident Learning algorithms to identify and prune instances where human labels violently contradict the model's internal probability distributions. Utilizing advanced external-guidance frameworks like LAFT allows the model to learn via "soft labels" for truly noisy data, preventing the catastrophic memorization of incorrect boundary artifacts that destroy recall.
3. **Architectural Evolution via Ordinal Regression:** The standard utilization of categorical cross-entropy for scaled sentiment tasks must be abandoned in favor of topological mathematics. Implementing the Ordinal Log-Loss (OLL) or the rank-consistent CORAL framework within the Hugging Face SFTTrainer via a custom compute_loss_func forces the model to respect the geometric distance between sentiments. This approach heavily penalizes critical errors and strictly smooths the decision boundary.
4. **Security Threat Modeling and Boundary Defense:** The resolution of the neutral boundary is inextricably linked to the physical security of the model. The discovery that frontier LLMs require an invariant, absolute number of poisoned samples (approximately 250 documents) to develop an undetectable stealth backdoor necessitates the aggressive, automated monitoring of boundary cases. Attackers actively exploit the high variance and low confidence of neutral zones to hide adversarial triggers. Consequently,

robust ordinal modeling and confident data pruning are not merely statistical performance enhancements; they are foundational, critical defensive mechanisms against the next generation of data poisoning attacks.

Ultimately, the process of fine-tuning large language models for complex, subjective tasks requires abandoning the historical assumption that massive scale and sheer data volume will inherently resolve semantic ambiguity. It demands mathematically rigorous loss functions, algorithmically curated datasets, and an acute awareness of the vulnerabilities that hide in the grey areas of human language. By stabilizing the neutral boundary, developers secure both the analytical utility and the operational integrity of the deployed model in an increasingly adversarial landscape.

Works cited

1. NEUTRAL_CLASS_RESEARCH_BRIEF.md
2. The Ultimate Guide to Fine-Tune LLaMA 3, With LLM Evaluations - Confident AI, accessed April 26, 2026, https://www.confident-ai.com/blog/the-ultimate-guide-to-fine-tune-llama-2-with-llm-evaluations
3. Beginner's Guide: How to Fine-tune Llama 3.1 Ultra-Efficiently with Unsloth & Deploy to Hugging Face : r/LocalLLaMA - Reddit, accessed April 26, 2026, https://www.reddit.com/r/LocalLLaMA/comments/1es8cuc/beginners_guide_how_to_finetune_llama_31/
4. LLM Data Poisoning Statistics 2026: Critical Facts You Must Know Now - SQ Magazine, accessed April 26, 2026, https://sqmagazine.co.uk/llm-data-poisoning-statistics/
5. Poisoning Attacks on LLMs Require a Near-constant Number of Poison Samples - arXiv, accessed April 26, 2026, https://arxiv.org/abs/2510.07192
6. Poisoning Attacks on LLMs - Emergent Mind, accessed April 26, 2026, https://www.emergentmind.com/topics/poisoning-attacks-on-llms
7. Parameter-Efficient Fine-Tuning of Llama 3.1: A Comprehensive Guide - Medium, accessed April 26, 2026, https://medium.com/@govindarajpriyanthan/parameter-efficient-fine-tuning-of-llama-3-1-a-comprehensive-guide-bed38d232285
8. Unsloth Challenge 3 submission - Medium, accessed April 26, 2026, https://medium.com/@indosambhav/unsloth-challenge-3-submission-a460d9b29f20
9. Sentiment Classification of Amazon Product Reviews Based on Machine and Deep Learning Techniques: A Comparative Study - MDPI, accessed April 26, 2026, https://www.mdpi.com/1999-5903/18/3/138
10. (PDF) Sentiment Classification of Amazon Product Reviews Based on Machine and Deep Learning Techniques: A Comparative Study - ResearchGate, accessed April 26, 2026, https://www.researchgate.net/publication/401652097_Sentiment_Classification_of_Amazon_Product_Reviews_Based_on_Machine_and_Deep_Learning_Technique

s_A_Comparative_Study
11. Optimizing BERT for Sentiment Classification of Amazon Product Reviews: A Study on Class Imbalance and Misclassification Analysis - Tokyo City University, accessed April 26, 2026, https://research.tcu.ac.jp/en/publications/optimizing-bert-for-sentiment-classification-of-amazon-product-re/
12. Evaluating Large Language Models for Sentiment Analysis and Hesitancy Analysis on Vaccine Posts From Social Media: Qualitative Study - PMC, accessed April 26, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC12526656/
13. Sentiment Analysis: Why Your Model Misses Sarcasm - Data Annotation Company, accessed April 26, 2026, https://labelyourdata.com/articles/natural-language-processing/sentiment-analysis
14. How to Perform Ordinal Regression / Classification in PyTorch | Towards Data Science, accessed April 26, 2026, https://towardsdatascience.com/how-to-perform-ordinal-regression-classification-in-pytorch-361a2a095a99/
15. Bias in the Loop: Auditing LLM-as-a-Judge for Software Engineering - arXiv, accessed April 26, 2026, https://arxiv.org/html/2604.16790v1
16. LLM-as-a-Judge: When to Use Reasoning, CoT, and Explanations | by Aparna Dhinakaran | Data Science Collective | Medium, accessed April 26, 2026, https://medium.com/data-science-collective/llm-as-a-judge-when-to-use-reasoning-cot-and-explanations-964ad82ebc3d
17. LLM-as-a-Judge: A Practical Guide | Towards Data Science, accessed April 26, 2026, https://towardsdatascience.com/llm-as-a-judge-a-practical-guide/
18. LLM-as-a-judge: a complete guide to using LLMs for evaluations - Evidently AI, accessed April 26, 2026, https://www.evidentlyai.com/llm-guide/llm-as-a-judge
19. Beyond Stars: Bridging the Gap Between Ratings and Review Sentiment with LLM - arXiv, accessed April 26, 2026, https://arxiv.org/html/2509.20953v1
20. A Survey of Scaling in Large Language Model Reasoning - arXiv, accessed April 26, 2026, https://arxiv.org/html/2504.02181v2
21. Taming LLM Outputs: Your Guide to Structured Text Generation - Dataiku, accessed April 26, 2026, https://www.dataiku.com/stories/blog/your-guide-to-structured-text-generation
22. Creating Toxicity Detection Using LLM-as-a-Judge: A Guide and Best Practices, accessed April 26, 2026, https://ggx.corridorplatforms.com/blog/llm-as-a-judge-comprehensive-guide
23. OpinioRAG: Towards Generating User-Centric Opinion Highlights from Large-scale Online Reviews - arXiv, accessed April 26, 2026, https://arxiv.org/html/2509.00285v1
24. New LLM Evaluation Templates For Label Studio, accessed April 26, 2026, https://labelstud.io/blog/new-llm-evaluation-templates-for-label-studio/
25. A Systematic Survey of Automatic Prompt Optimization Techniques - ACL Anthology, accessed April 26, 2026, https://aclanthology.org/2025.emnlp-main.1681.pdf

26. [1911.00068] Confident Learning: Estimating Uncertainty in Dataset Labels - arXiv, accessed April 26, 2026, https://arxiv.org/abs/1911.00068
27. Cleanlab - AWS Marketplace, accessed April 26, 2026, https://aws.amazon.com/marketplace/seller-profile?id=87d98359-d6ba-4fca-bdd6-b45ad46110dd
28. classification - cleanlab, accessed April 26, 2026, https://docs.cleanlab.ai/v2.0.0/cleanlab/classification.html
29. Cleanlab's open-source library is the standard data-centric AI package for data quality and machine learning with messy, real-world data and labels. - GitHub, accessed April 26, 2026, https://github.com/cleanlab/cleanlab
30. Annotate text data using Active Learning with Cleanlab - Hugging Face, accessed April 26, 2026, https://huggingface.co/learn/cookbook/annotate_text_data_transformers_via_active_learning
31. Detecting Issues in a Text Dataset with Cleanlab - Hugging Face, accessed April 26, 2026, https://huggingface.co/learn/cookbook/issues_in_text_dataset
32. Noise-Robust Fine-Tuning of Pretrained Language ... - ACL Anthology, accessed April 26, 2026, https://aclanthology.org/2023.findings-emnlp.834.pdf
33. Noise-Robust Fine-Tuning of Pretrained Language Models via External Guidance, accessed April 26, 2026, https://openreview.net/forum?id=DSmHC8bi3j¬eId=tojwwda0SS
34. Soft Labels for Ordinal Regression - CVF Open Access, accessed April 26, 2026, https://openaccess.thecvf.com/content_CVPR_2019/papers/Diaz_Soft_Labels_for_Ordinal_Regression_CVPR_2019_paper.pdf
35. [1901.07884] Rank consistent ordinal regression for neural networks with application to age estimation - arXiv, accessed April 26, 2026, https://arxiv.org/abs/1901.07884
36. Predicting Ratings And Rankings Using PyTorch Lightning - Sebastian Raschka, accessed April 26, 2026, https://sebastianraschka.com/pdf/slides/2022-07-scipy-corn.pdf
37. Deep Neural Networks for Rank-Consistent Ordinal Regression Based On Conditional Probabilities - arXiv, accessed April 26, 2026, https://arxiv.org/html/2111.08851v5
38. Raschka-research-group/coral-pytorch: CORAL and CORN implementations for ordinal regression with deep neural networks. - GitHub, accessed April 26, 2026, https://github.com/Raschka-research-group/coral-pytorch
39. CORAL CNN model for image data (MNIST dataset) - coral_pytorch, accessed April 26, 2026, https://raschka-research-group.github.io/coral-pytorch/tutorials/pure_pytorch/CORAL_mnist/
40. A Simple Log-based Loss Function for Ordinal Text ... - ACL Anthology, accessed April 26, 2026, https://aclanthology.org/2022.coling-1.407.pdf
41. fpsigma/trl-slime: Train transformer language models with reinforcement learning. - GitHub, accessed April 26, 2026, https://github.com/fpsigma/trl-slime
42. SFT Trainer · Hugging Face, accessed April 26, 2026,

https://huggingface.co/docs/trl/main/en/sft_trainer
43. SFTTrainer Loss function - Beginners - Hugging Face Forums, accessed April 26, 2026, https://discuss.huggingface.co/t/sfttrainer-loss-function/80549
44. sergeipetrov/transformers-diffusers-docs-raw · Datasets at Hugging Face, accessed April 26, 2026, https://huggingface.co/datasets/sergeipetrov/transformers-diffusers-docs-raw/viewer/default/train
45. Practical Attacks on LLMs: Full Guide to LLM Attacks - Iterasec, accessed April 26, 2026, https://iterasec.com/blog/practical-attacks-on-llms/
46. On The Dangers of Poisoned LLMs In Security Automation - arXiv, accessed April 26, 2026, https://arxiv.org/html/2511.02600v1
47. LLM04:2025 Data and Model Poisoning - OWASP Gen AI Security Project, accessed April 26, 2026, https://genai.owasp.org/llmrisk/llm042025-data-and-model-poisoning/
48. Introduction to Data Poisoning: A 2026 Perspective | Lakera – Protecting AI teams that disrupt the world., accessed April 26, 2026, https://www.lakera.ai/blog/training-data-poisoning
49. Data Poisoning: a threat to LLM's Integrity and Security - RiskInsight, accessed April 26, 2026, https://www.riskinsight-wavestone.com/en/2024/10/data-poisoning-a-threat-to-llms-integrity-and-security/
50. A Systematic Review of Poisoning Attacks Against Large Language Models (LLM) - ODU Digital Commons, accessed April 26, 2026, https://digitalcommons.odu.edu/cgi/viewcontent.cgi?article=1145&context=covacci-undergraduateresearch
51. Poisoning Attacks on LLMs Require a Near-constant Number of Poison Samples - arXiv, accessed April 26, 2026, https://arxiv.org/pdf/2510.07192
52. LLMs may be more vulnerable to data poisoning than we thought | The Alan Turing Institute, accessed April 26, 2026, https://www.turing.ac.uk/blog/llms-may-be-more-vulnerable-data-poisoning-we-thought
53. A small number of samples can poison LLMs of any size - Anthropic, accessed April 26, 2026, https://www.anthropic.com/research/small-samples-poison
54. Abstractive - Nicholas Carlini, accessed April 26, 2026, https://nicholas.carlini.com/writing/2019/advex_papers_with_abstracts.txt
55. Track: Poster Session 6 & Exhibit Hall - CVPR 2026, accessed April 26, 2026, https://cvpr.thecvf.com/virtual/2024/session/32088
56. BrowseSafe: Understanding and Preventing Prompt Injection Within AI Browser Agents, accessed April 26, 2026, https://arxiv.org/html/2511.20597v1
57. A Systematic Review of Poisoning Attacks Against Large Language Models - arXiv, accessed April 26, 2026, https://arxiv.org/html/2506.06518v1
58. A Survey of Backdoor Attacks and Defenses on Large Language Models: Implications for Security Measures - arXiv, accessed April 26, 2026, https://arxiv.org/html/2406.06852v3
59. Adversarial Training for Defense Against Label Poisoning Attacks - OpenReview,

accessed April 26, 2026, https://openreview.net/forum?id=UlpkHciYQP
60. Cost-Minimized Label-Flipping Poisoning Attack to LLM Alignment - AAAI Publications, accessed April 26, 2026, https://ojs.aaai.org/index.php/AAAI/article/view/41087/45048
61. Emerging Threat Vectors: How Malicious Actors Exploit LLMs to Undermine Border Security, accessed April 26, 2026, https://www.preprints.org/manuscript/202508.0668
62. Emerging Threat Vectors: How Malicious Actors Exploit LLMs to Undermine Border Security, accessed April 26, 2026, https://www.mdpi.com/2673-2688/6/9/232